# 实验四　传输层实验

## 1　实验内容



实验四
内容简介

传输层实验主要包括 TCP 和 UDP 两个部分，重点是 TCP 拥塞控制实验，为了达到定量化分析 TCP 拥塞控制算法和机制的目的，TCP 实验中的主机都使用 Linux 系统，通过修改 Linux 内核程序，将 Linux 内核中 TCP 全部过程的相关状态和参数的实时数据全部记录下来，并导出到文件中进行分析。同时，还通过配置网络设备以设定不同的网络带宽，改变 TCP 测试程序相关参数，以及人为制造多种网络拥塞状况（如接收端休眠、断开连接、人为丢包）等方式，帮助对相应的拥塞控制算法进行实验分析。本实验的操作和配置不复杂，以分析实验数据为主。具体实验内容如下。

(1) TCP 基本分析。

(2) TCP 的拥塞控制。

(3) UDP 分析。

(4) 简单的 Socket 编程。

## 2　TCP 基本分析



TCP 基本
分析

### 2.1　实验目的

理解 TCP 报文首部格式和字段的作用，TCP 连接的建立和释放的过程，TCP 数据传输中编号与确认的过程。

### 2.2　实验内容

应用 TCP 应用程序传输文件，截取 TCP 报文，分析 TCP 报文首部信息、TCP 连接的建立和释放过程、TCP 数据的编号与确认机制。

CS 扫描全能王

## 2.3 实验原理

TCP是传输控制协议(Transfer Control Protocol)的简称。TCP工作在网络层协议之上,是一个面向连接的、端到端的、可靠的传输层协议。

### 2.3.1 TCP的报文格式

如图4-1所示,TCP的报文段分为首部和数据两部分。

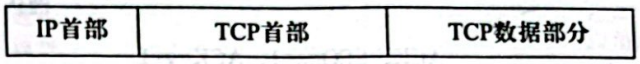



图4-1 TCP报文段的总体结构

TCP的报文段首部又分为固定部分和选项部分,如图4-2所示,固定部分共20字节,主要字段有源端口、目的端口、序号、确认号、数据偏移、保留、码元比特、窗口、校验和、紧急指针、选项和填充字段,各字段的意义参见参考文献1。正是这些字段作用的有机结合,实现了TCP的全部功能。

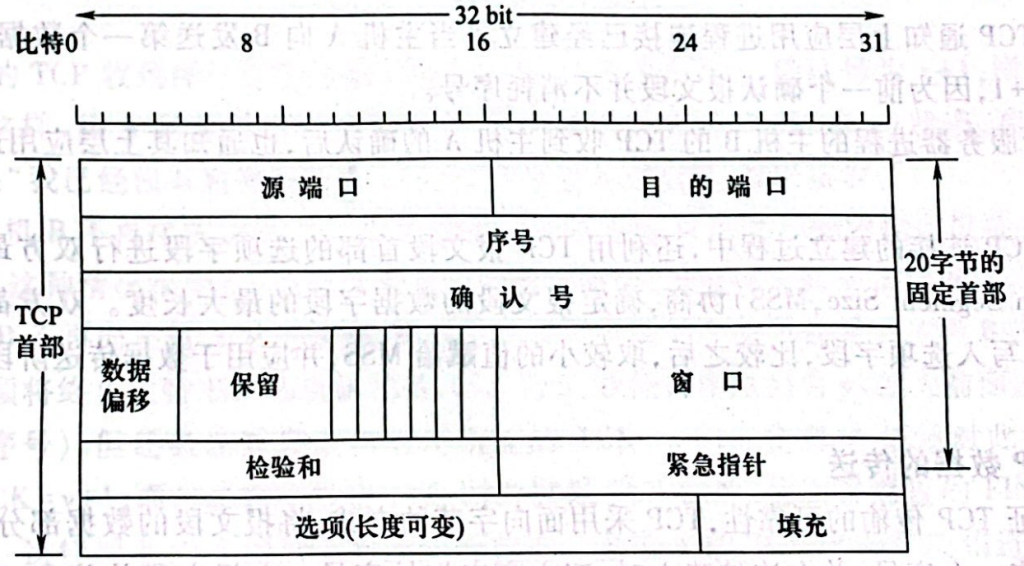



图4-2 TCP报文段的首部

TCP采用运输连接的方式传送TCP报文,运输连接包括连接建立、数据传送和连接释放3个阶段。

### 2.3.2 TCP连接的建立

TCP连接的建立采用了3次握手(Three-way Handshake)方式。

首先,主机A的TCP向主机B的TCP发出连接请求报文段,其首部中的同步比特SYN应置1,同时选择一个序号x,表明在后面传送数据时的第一个数据字节的序号是x+1,如图4-3所示。

CS 扫描全能王

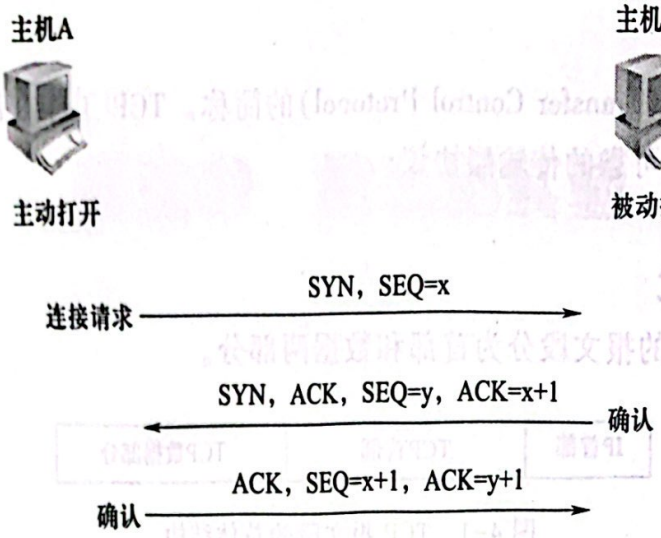



图 4-3　TCP 连接的建立

然后，主机 B 的 TCP 收到连接请求报文段后，若同意，则发回确认。在确认报文段中应将 SYN 和 ACK 都置 1，确认号应为 x+1，同时也为自己选择一个序号 y。

最后，主机 A 的 TCP 收到 B 的确认后，要向 B 给出确认，其 ACK 置 1，确认号为 y+1，而自己的序号为 x+1。TCP 的标准规定，SYN 置 1 的报文段要消耗掉一个序号。同时，运行客户进程的主机 A 的 TCP 通知上层应用进程连接已经建立。当主机 A 向 B 发送第一个数据报文段时，其序号仍为 x+1，因为前一个确认报文段并不消耗序号。

当运行服务器进程的主机 B 的 TCP 收到主机 A 的确认后，也通知其上层应用进程连接已经建立。

另外，TCP 连接的建立过程中，还利用 TCP 报文段首部的选项字段进行双方最大报文段长度（Maximum Segment Size，MSS）协商，确定报文段的数据字段的最大长度。双方都将自己能够支持的 MSS 写入选项字段，比较之后，取较小的值赋给 MSS，并应用于数据传送阶段。

### 2.3.3　TCP 数据的传送

为了保证 TCP 传输的可靠性，TCP 采用面向字节的方式，将报文段的数据部分进行编号，每一个字节对应一个序号，并在连接建立时，双方商定初始序号。在报文段首部中，序号字段和长度字段可以确定发送方传送数据的每一个字节的序号，确认号字段则表示接收方希望下次收到的数据的第一个字节的序号，即表示这个序号之前的数据字节均已收到。这样，既做到了可靠传输，又做到了全双工通信。

当然，数据传送阶段有很多非常复杂的问题和情况，如流量控制、拥塞控制、重传控制等，将在下一部分介绍。

### 2.3.4　TCP 连接的释放

在数据传输结束后，通信的双方都可以发出释放连接的请求。TCP 连接的释放采用所谓 4

CS 扫描全能王

次握手方式。

首先，设图4-4中的主机A的应用进程先向其TCP发出连接释放请求，并且不再发送数据。TCP通知对方要释放从A到B这个方向的连接，将发往主机B的TCP报文段首部的终止比特FIN置1，其序号x等于前面已传过的数据的最后一个字节的序号加1。

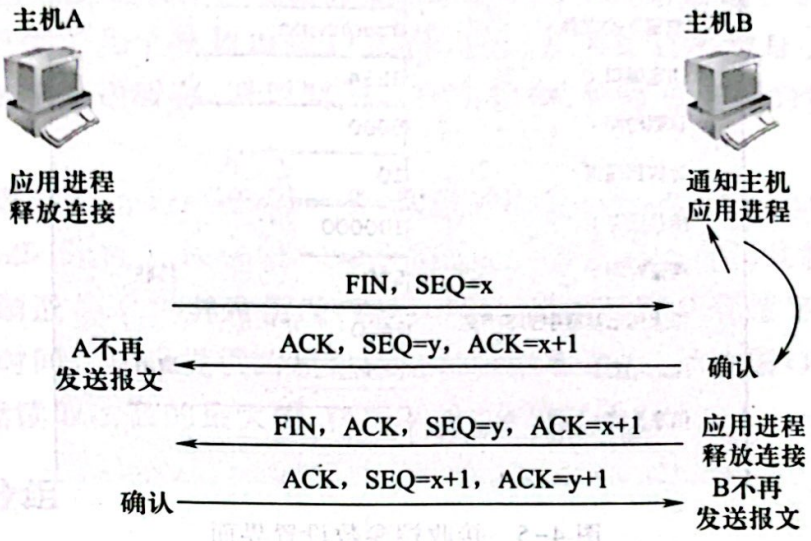



图4-4 TCP连接释放过程

主机B的TCP收到释放连接通知后即发出确认，其序号为y，确认号为x+1，同时通知高层应用进程。这样，从A到B的连接就释放了，连接处于半关闭(half-close)状态，相当于主机A向主机B说："我已经没有数据要发送了，但你如果还发送数据，我仍接收。"

此后，主机B不再接收主机A发来的数据。但若主机B还有一些数据要发往主机A，则可以继续发送(这种情况很少)。主机A只要正确收到数据，仍应向主机B发送确认。

若主机B不再向主机A发送数据，其应用进程就通知TCP释放连接。主机B发出的连接释放报文段必须将终止比特FIN和确认比特ACK置1，并使其序号仍为y(因为前面发送的确认报文段不消耗序号)，但还必须重复上次已发送过的ACK=x+1。主机A必须对此发出确认，将ACK置1，ACK=y+1，而自己的序号是x+1，因为根据TCP标准，前面发送过的FIN报文段要消耗一个序号。这样把B到A的反方向连接释放掉。主机A的TCP再向其应用进程报告，整个连接已经全部释放。

### 2.3.5 TcpTest程序介绍

Linux系统下的TcpTest程序是用Java编写的基于TCP的文件传输程序。程序主要用到了Socket、ServerSocket、FileOutputStream、FileInputStream、DataOutputStream、DataInputStream等几个类。实现从发送端读取一个文件，写入套接字，再从接收端套接字读取数据，写入指定文件。程序界面如图4-5、图4-6所示，通过适当选取参数，程序具有改变发送、接收缓存；启用、禁用Nagle算法；设置接收端休眠等功能。程序中的时间单位都是毫秒。

CS 扫描全能王

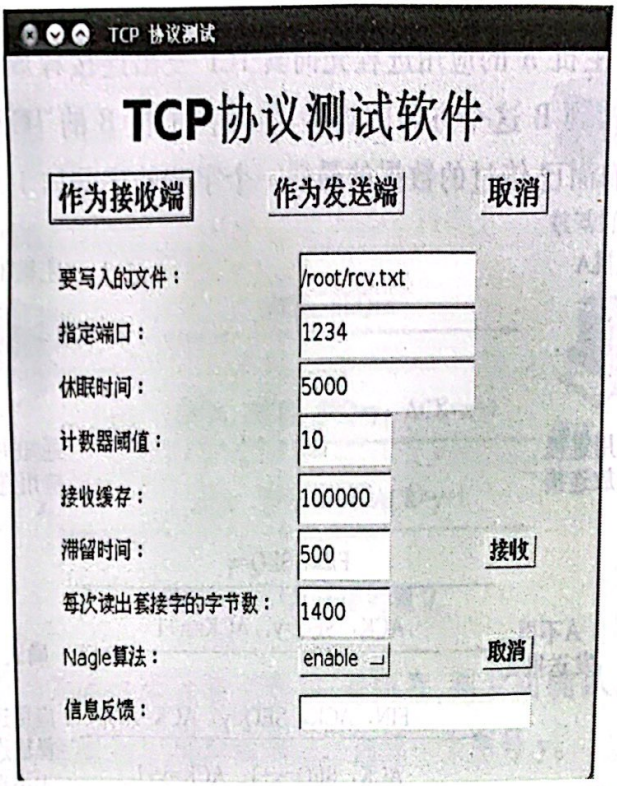



图 4-5　接收端参数设置界面

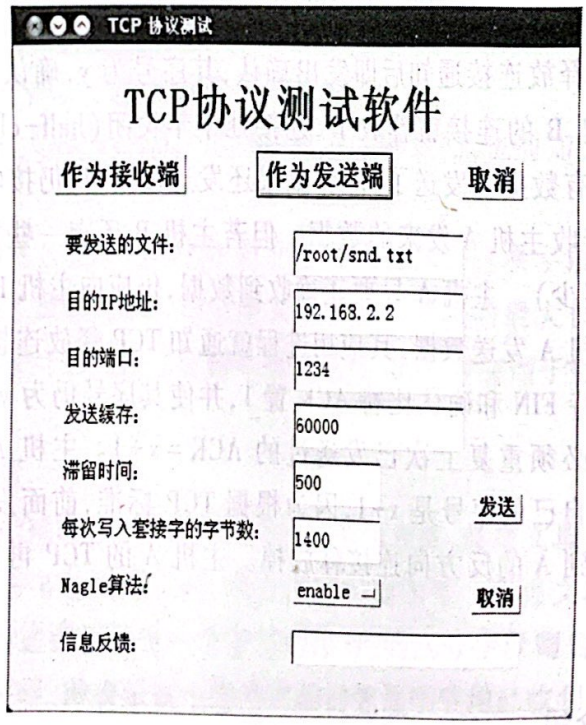



图 4-6　发送端参数设置界面

打开 TcpTest 程序的方法如下。

方法 1(图形界面方式):双击桌面上的 tcptest 脚本,在弹出的对话框中选择“在终端中运行”或“运行”命令。

方法 2(命令方式):按 Ctrl+Alt+T 键打开终端,在终端中输入以下命令:

CS 扫描全能王

```
root@qjl-desktop:~#    cd  /root/TCPTest/
root@qjl-desktop:~/TCPTest#    /root/jre/bin/java  TcpTest
```

发送端作为客户端,使用 Socket 类,构造方法为 Socket(InetAddress address, int port),两个参数分别为目的地址和目的端口。使用 setSendBufferSize()、setTcpNoDelay()、setSoTimeout()、setSoLinger()方法来设定发送缓存、Nagle 算法、超时时间和滞留时间。为了达到分析 TCP 工作机理的目的,程序提供了几个参数由用户选择,它们分别是目的地址、目的端口、每次写入套接字的字节数、套接字发送缓存、滞留时间。在实验中,将通过不同的参数设定来观察和分析 TCP。

接收端作为服务器端使用 ServerSocket 类,构造方法为 ServerSocket(int port),参数是本地端口。使用 setReceiveBufferSize()、accept()、setSoLinger()方法来设定接收缓存、监听指定端口和滞留时间。为了人为制造 TCP 的异常情况,程序中可以通过设定从套接字读取数据的次数来使接收端程序休眠一段时间。程序提供的用户接口包括以下参数:接收端口、接收缓存、每次读出套接字的字节数、连续读取次数的最大值、休眠时间、滞留时间。

## 2.4 实验环境与分组

(1) 路由器和交换机各 2 台,标准网线 4 条,Console 线 4 条,计算机 4 台。

(2) 每 4 名学生为一组,其中每 2 名学生分为一小组,每小组共同配置 1 台路由器和交换机。

(3) 本实验 PC 采用的是 Linux 的 Ubuntu 10.0 操作系统。

## 2.5 实验组网

实验组网如图 4-7 所示。

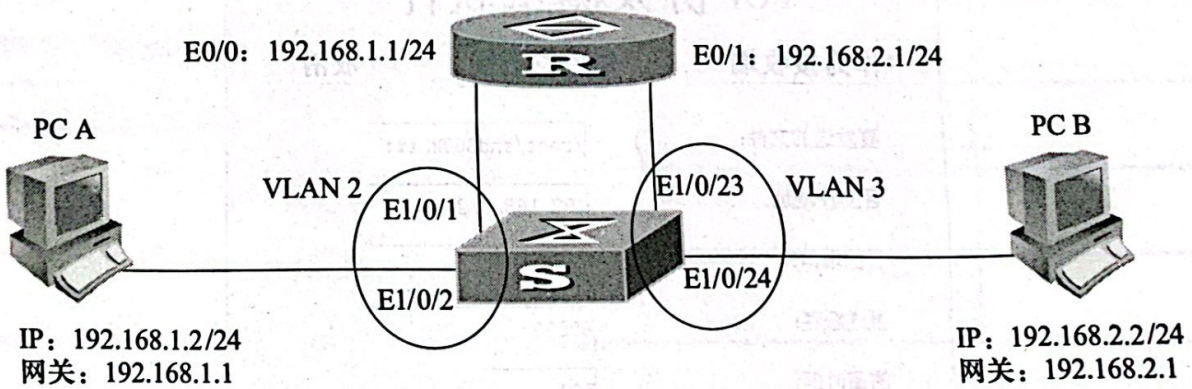



图 4-7 TCP 分析组网图

注:由于计算机与路由器需要用交叉线连接,如果使用直连线需要通过交换机转换一下,方法就是用标准网线把 P( 交换机上,然后从交换机上另外一个端口连到路由器相应端口,并划分相应的 VLAN 即可。

CS 扫描全能王

## 2.6 实验步骤

**步骤 1** 按照图 4-7 进行组网,并配置路由器和交换机。(在“终端命令行”中执行 Lin
级终端命令,minicon)

**步骤 2** PC A 和 PC B 均进入 Linux 操作系统(用户名:root,密码:network),启动 Wire
软件进行报文截获。

**步骤 3** 如图 4-8 和图 4-9 所示,在 PC A 和 PC B 上分别运行 TcpTest 程序的发送和
模块:

```
root@qjl-desktop:~#    cd  /root/TCPTest/
root@qjl-desktop:~/TCPTest#    /root/jre/bin/java  TcpTest
```

传送一个 300 KB 的文件,文件传输完毕后,保存截获报文为文件“send1-学号”和“receiv
学号”;上传至 FTP 服务器的“网络实验\传输层实验”目录下。

图 4-9 中的“休眠时间”表示接收端进程每次休眠的时间;“计数器阈值”表示每次进行休
前从套接字读取数据的次数。这里把它们都设成 0,建立正常的 TCP 连接,分析文件传递
过程。

这里,TCP 的连接和建立采用的是:_____方式,PC A 是_____,PC B 是_____。
先单击“发送”按钮再单击“接收”按钮,会出现什么问题? 为什么?

**步骤 4** 分析截获报文中数据发送部分的第一条 TCP 报文及其确认报文,将报文中的字
值填写在预习报告的表格中。

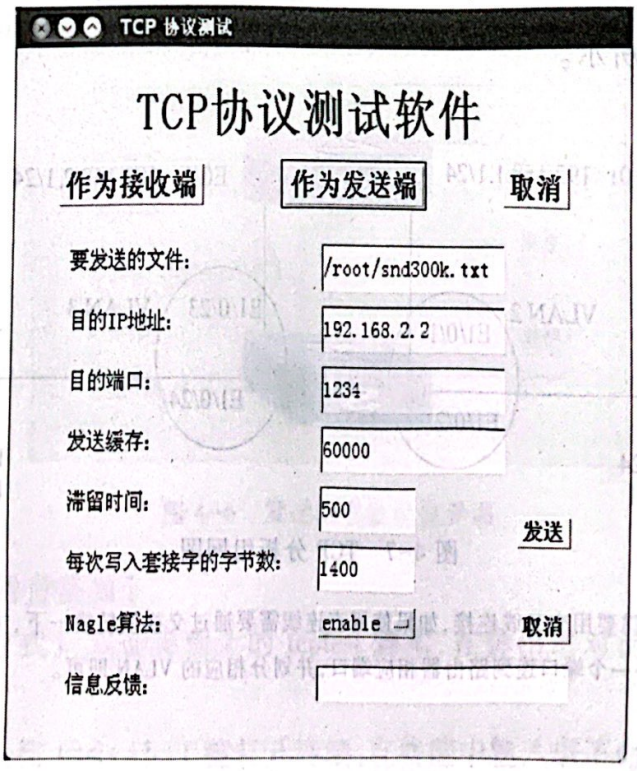



图 4-8 发送端参数设置

CS 扫描全能王

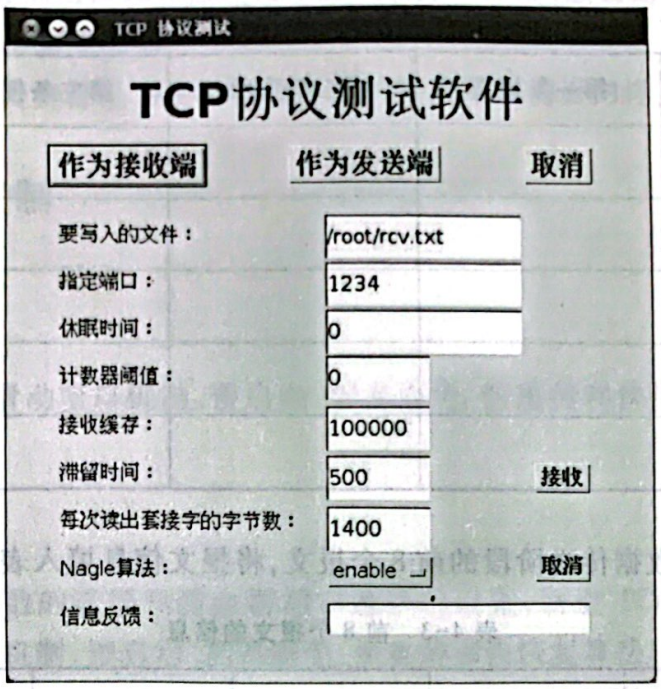



图 4-9 接收端参数设置

**步骤 5** 结合预习报告，分析 TCP 连接的建立过程，根据 TCP 建立过程的 3 个报文填写表 4-1。

**表 4-1 TCP 建立过程的 3 个报文**

| 字段名称 | 第一条报文 | 第二条报文 | 第三条报文 |
|---|---|---|---|
| 报文序号 | | | |
| Sequence Number | | | |
| Acknowledgement Number | | | |
| ACK | | | |
| SYN | | | |

**步骤 6** TCP 连接建立时，其报文首部与其他 TCP 报文不同，有一个“Option”字段，它的作用是什么，有什么内容？值为多少？结合 IEEE 802.3 协议规定的以太网最大帧长度分析此数据是怎样得出的。

**步骤 7** 结合预习报告，分析 TCP 连接的释放过程，选择 TCP 连接撤销的 4 个报文，将报文信息填入表 4-2。

CS 扫描全能王

表 4-2　TCP 连接撤销的 4 个报文

| 字段名称 | 第一条报文 | 第二条报文 | 第三条报文 | 第四条报文 |
|---|---|---|---|---|
| 报文序号 | | | | |
| Sequence Number | | | | |
| Acknowledgement Number | | | | |
| ACK | | | | |
| FIN | | | | |

**步骤 8**　分析 TCP 数据传送阶段的前 8 个报文，将报文信息填入表 4-3。

表 4-3　前 8 个报文的信息

| 报文序号 | 报文种类（发送/确认） | 序号字段 | 确认号字段 | 数据长度 | 被确认报文序号 | 窗口 |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

请写出 TCP 数据部分长度的计算公式。数据传送阶段第一个报文的序号字段值是否等于连接建立时第三个报文的序号？

## 2.7　实验总结

在实验中，通过分析截获的 TCP 报文首部信息，可以看到首部中的序号、确认号等字段是 TCP 的可靠连接的基础。然后，通过分析 TCP 连接的 3 次握手建立和 4 次握手的释放过程，理

解了 TCP 连接建立和释放的机制和控制比特字段的作用。最后,通过对数据传送阶段报文的初步分析,了解数据的编号和确认机制。

总之,TCP 中的各项设置都是为了在数据传输时提供可靠的面向连接的服务。

# 3 TCP 的拥塞控制

## 3.1 实验目的

理解和体会 TCP 的滑动窗口机制、慢启动、拥塞避免、快重传和快恢复算法,以及坚持定时器的作用。

## 3.2 实验内容

通过 TcpTest 程序参数的选择和路由器端口速率的设置,制造 TCP 传输的不同环境,截取 TCP 报文,分析滑动窗口机制、慢启动、拥塞避免、快重传和快恢复算法,以及坚持定时器的作用。具体内容如下。

(1) 滑动窗口机制和窗口侦查机制分析。

(2) 糊涂窗口综合征和 Nagle 算法分析。

(3) 慢启动、拥塞避免及拥塞处理和超时与重传机制分析。

(4) 快重传和快恢复算法分析。

## 3.3 实验原理

TCP 的拥塞控制和流量控制是一个比较复杂的问题,它包括发送端发送报文的大小和报文的时机,接收端发送确认和窗口大小的策略。同时还要兼顾不同网络的具体情况,算法要具有一定的自适应性,在保证可靠传输的同时,尽量提高传输效率。

这里主要对目前公认的比较行之有效的一些拥塞控制和流量控制算法进行介绍和验证,主要有 TCP 的滑动窗口机制、TCP 的糊涂窗口综合征和 Nagle 算法分析、网络拥塞的处理、TCP 超时与重传、TCP 的窗口探查技术、TCP 的快重传和快恢复。

### 3.3.1 TCP 的滑动窗口机制

为了提高报文段的传输速率,TCP 采用大小可变的滑动窗口进行流量控制。窗口大小的单位是字节。发送窗口在连接建立时由双方商定,但在通信过程中,接收端可根据自己的接收缓存的大小,随时动态地调整发送端的发送窗口的上限值。这就是接收端窗口 rwnd(receiver window),这个值被放在接收端发送的 TCP 报文段首部的窗口字段中。



TCP 的滑
窗口机

CS 扫描全能王

同时，发送端根据其对当前网络拥塞程度的估计而确定的窗口值，叫做拥塞窗口 cwnd（congestion window）。其大小与网络的带宽和时延密切相关。

发送端设置的当前能够发送数据量的大小叫做发送窗口，发送窗口的上限值由下面公式确定：

$$\text{发送窗口的上限值} = \text{Min}[\text{cwnd}, \text{rwnd}]$$

rwnd 由接收端根据其接收缓存确定，发送端确定 cwnd 比较复杂，详细情况在慢启动和拥塞避免一节中叙述。

发送窗口的左边沿对应已发送数据中被确认的最高序号+1，其右边沿对应左边沿的序号加上发送窗口的大小。在数据传输的过程中，这个发送窗口不时地向右移动构成了滑动窗口。窗口的两个边沿的相对运动增加或减少了窗口的大小。这里使用 3 个术语来描述窗口左右边沿的运动。

（1）当窗口左边沿向右边沿靠近时，称之为窗口合拢。这种现象发生在数据被发送和确认时。如果窗口的左边沿与右边沿重合，则称其为一个零窗口，此时发送方不能发送任何数据。

（2）当窗口右边沿向右移动时将允许发送更多的数据，称之为窗口张开。这种现象发生在另一端的接收进程读取已经确认的数据并释放了 TCP 的接收缓存时。

（3）当右边沿向左移动时，称之为窗口收缩。这种情况一般不会发生，但是 TCP 必须能够在某一端产生这种情况时进行处理。

### 3.3.2　TCP 的糊涂窗口综合征和 Nagle 算法



TCP 的糊涂窗口综合征和 Nagle 算法

TCP 的流量控制方案是基于窗口的，有可能会出现一种被称为“糊涂窗口综合征”的状况。其中一种情况是，如果接收方处理较慢，并且每次从其接收缓存取走很少量数据就通告这个很小的窗口，而不是等到有较大的窗口时才通告；发送方得到这个很小的接收窗口后，立即按照这个窗口大小组成一个 TCP 报文段发送出去，而不是等待接收窗口变大后以便发送一个大的报文段。如此往复，会导致网络的传输效率降低。

对于糊涂窗口综合征的现象，发送和接收双方均可以采取措施加以避免。

发送端比较有效的方法是采用 Nagle 算法。该算法主要是，在连接建立开始发送数据时，立即按序发送缓存中的数据（必须小于或等于 MSS），在已经传输的数据还未被确认的情况下，后续数据的发送由数据是否足以填满发送缓存的一半或一个最大报文段长度决定。

接收端采用推迟确认技术，对收到的报文段进行确认和通告窗口的前提条件是，接收缓存的可用空间至少达到总空间的一半或者达到最大报文长度。如果条件不满足，则推迟发送确认和窗口通告。

总之，避免糊涂窗口综合征的总的原则是，接收端避免通告小窗口，发送端尽量将数据组成较大的报文段发送出去。

CS 扫描全能王

### 3.3.3 TCP 的慢启动和拥塞避免



TCP 的慢启动和拥塞避免

为了保证网络平稳高效地运行，防止网络流量的剧烈起伏振荡。1997 年公布的因特网建议标准[RFC 2001]提出了慢启动(Slow-start)和拥塞避免算法(Congestion Avoidance)。

慢启动算法的原理是，在主机开始发送数据时，采用试探的方式，由小到大逐渐增大发送端的拥塞窗口数值。通常是在一开始 cwnd 应设置为不超过 2×MSS(最大报文段)个字节，在每收到一个对新的报文段的确认后，拥塞窗口至多增加 1 个 MSS 的数值，使分组注入网络的速率比较合理。

拥塞避免算法是使发送端的拥塞窗口 cwnd 值在每收到一个非重复的 ACK 报文后，增加一个 SMSS×SMSS/cwnd 的大小。也就是当发送方每收到 cwnd 个非重复的 ACK 报文，cwnd 加 1。可以论证，基本上是时间每经过一个 RTT，cwnd 就会加 1。其中 SMSS 是发送端的 MSS。(详见 RFC2001)

慢启动与拥塞避免算法相比较，拥塞窗口增加的方式分别是指数方式和线性方式。慢启动算法使发送端在发送数据的开始阶段逐步增加注入网络的分组数，但随着拥塞窗口按指数方式快速增长，势必会引起网络拥塞。需要在网络拥塞之前，将拥塞窗口的增长速率降下来，也就是将慢启动算法切换到拥塞避免算法。因此，需要设置一个慢启动门限变量 ssthresh，利用 ssthresh 得到慢启动和拥塞避免的综合算法：

当 cwnd<ssthresh 时，使用慢启动算法。

当 cwnd>ssthresh 时，使用拥塞避免算法。

当 cwnd=ssthresh 时，既可以使用慢启动算法，也可以使用拥塞避免算法。

### 3.3.4 网络拥塞的处理



网络拥塞的处理

网络拥塞是指发送端没有按时收到确认报文或者收到了重复的确认报文。

在任何时候，只要发送端发现网络拥塞，就立即进行网络拥塞的处理，RFC 2001 给出如下公式设置慢启动门限值：

$$ssthresh = \max(cwnd/2, 2\times MSS)$$

以及设置拥塞窗口：cwnd=1。

然后，重新执行上一小节所述的慢启动和拥塞避免的综合算法。

这样，能够迅速地减少主机发送到网络中的分组数，使得发生拥塞的主机或者路由器有时间把队列中的积压分组处理完毕。

CS 扫描全能王

### 3.3.5　TCP 的超时与重传


TCP 的超时与重传

超时与重传机制是 TCP 中最重要和最复杂的技术之一。发送端每发送一个报文段，TCP 为其保留一个副本，设定一个定时器并等待确认信息。如果定时器超时，而发送的报文段中的数据仍未得到确认，则重传这一报文段。由此可见，定时器是重传数据的关键。

针对网络环境的复杂性，TCP 采用了一种自适应算法，提出超时重传时间应略大于平均往返时延 RTT，而 RTT 是根据各个报文段的往返时延样本的加权平均得出的。如何比较精确地估计 RTT 的值，Karn 算法是目前公认的效果较好的算法。

Karn 算法提出在计算平均往返时延 RTT 时，不计算发生过报文段重传的往返时延样本；同时报文段每重传一次，相应增大重传时间：

$$\text{新的重传时间} = \gamma \times \text{旧的重传时间}$$

其中，系数 $\gamma$ 的典型值是 2。并且，当不再发生报文段重传时，才根据报文段的往返时延更新 RTT 和重传时间的数值。

这样得出的平均往返时延 RTT 和重传时间就比较准确，并且实践证明，该方法比较合理和有效。

### 3.3.6　TCP 的窗口探查技术


TCP 的窗口探查技术

当接收端的接收缓存已满，不能继续接收数据时，需要向发送端发送一个窗口为 0 的通告报文。发送端接收到这个报文后，停止发送数据，等待新的窗口通告。如果接收端通过确认报文通告窗口，TCP 并不对这个确认报文进行确认，如果这个确认丢失了，则双方就有可能因为等待对方而使连接中止：接收方等待接收数据（因为它已经向发送方通告了一个非 0 的窗口），而发送方在等待允许它继续发送数据的窗口更新。为防止这种死锁情况的发生，发送方使用一个坚持定时器（Persist Timer）来周期性地向接收方查询，以便发现窗口是否已增大。这些从发送方发出的报文段称为窗口探查（Window Probe），窗口探查是包含一个字节的数据的报文段。

### 3.3.7　TCP 的快重传和快恢复


TCP 的快重传和快恢复

为了避免 TCP 因等待重传定时器超时而空闲较长时间，又提出了两个新的拥塞控制算法：快重传和快恢复。

快重传算法是指当发送端连续收到 3 个重复的 ACK 报文时，即可认为某一报文段丢失并且网络仍能够进行正常报文传输。因此，不必等待那个报文的定时器超时，而直接重传那个认为是丢失的报文段。即在某些情况下更早地重传被估计为丢失的报文段。

CS 扫描全能王

快恢复算法是慢启动算法的一个补充，它与快重传算法配合使用。根据 RFC 2001，其具体步骤如下。

(1) 当发送端收到连续 $n(n \geq 3)$ 个重复的 ACK，设置慢启动门限值：

$$ssthresh = max(cwnd/2, 2 * MSS)$$

同时，将 cwnd 设置为

cwnd = min[cwnd，已发送的报文数-已到达接收方但未正式确认的报文+重传的报文数+3]

(2) 如果发送窗口值还容许发送报文段，就按拥塞避免算法继续发送报文段。

(3) 若收到了确认新的报文段的 ACK，就将 cwnd 缩小到 ssthresh。

在采用快恢复算法时，慢启动算法只在 TCP 连接建立时才使用。

以上简要介绍了主要的 TCP 的拥塞控制算法，供实验预习参考。没有介绍到的内容(如 RED 算法等)，请参见参考文献 1。

## 3.4 实验环境与分组

(1) 路由器和交换机各 2 台，标准网线 4 条，Console 线 4 条，计算机 4 台。

(2) 每 4 名学生为一组，其中每 2 名学生分为一小组，每小组共同配置 1 台路由器和交换机。

(3) 本实验 PC 采用的是 Linux 的 Ubuntu 10.0 操作系统。

## 3.5 实验组网

实验组网如图 4-10 所示。

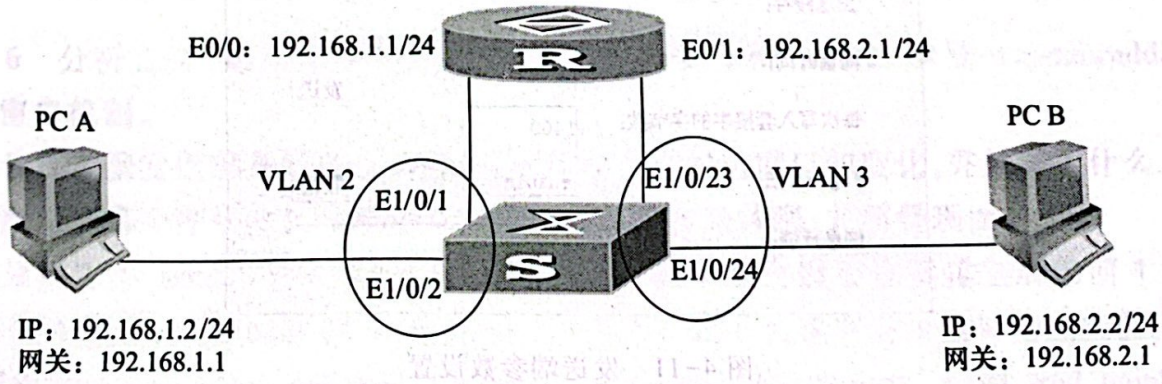



图 4-10 TCP 分析组网图

## 3.6 实验步骤(Linux 环境下的拥塞控制实验)

### 3.6.1 滑动窗口机制和窗口侦查机制分析

步骤 1 按照图 4-10 所示进行组网，确保组网正确和路由器、PC 接口 IP 地址配置正确。

CS 扫描全能王

**步骤 2**　所有 PC 进入 Linux 操作系统(用户名:root,密码:network)。打开所有 PC 上 Wireshark 软件,进行报文截获。注意:启动 Wireshark 建议最好使用 TCP 报文过滤。

**步骤 3**　PC A(PC C)(即发送端)在 Linux 下"终端命令行"中运行脚本来初始化"TCPConnection 实时监控模块":

```
root@qjl-desktop:~#     cd /root/TCPLog          //进入目录/root/TCPLog
root@qjl-desktop:~/TCPLog#     ./init.sh         //点斜杠表示当前目录
```

**步骤 4**　所有 PC 在 Linux 下打开一个新的"终端命令行",使用命令行方式启动 TcpTest 程序:

```
root@qjl-desktop:~#     cd  /root/TCPTest/
root@qjl-desktop:~/TCPTest#     /root/jre/bin/java  TcpTest
```

PC A(PC C)作为发送端,并选择一个 300 KB 的文件准备发送,PC B(PC D)作为接收端,参数设置如图 4-11 和图 4-12 所示(要发送的文件处改成/root/snd300k.txt)。检查参数设置无误后 PC B(PC D)单击"接收"按钮,然后 PC A(PC C)单击"发送"按钮。

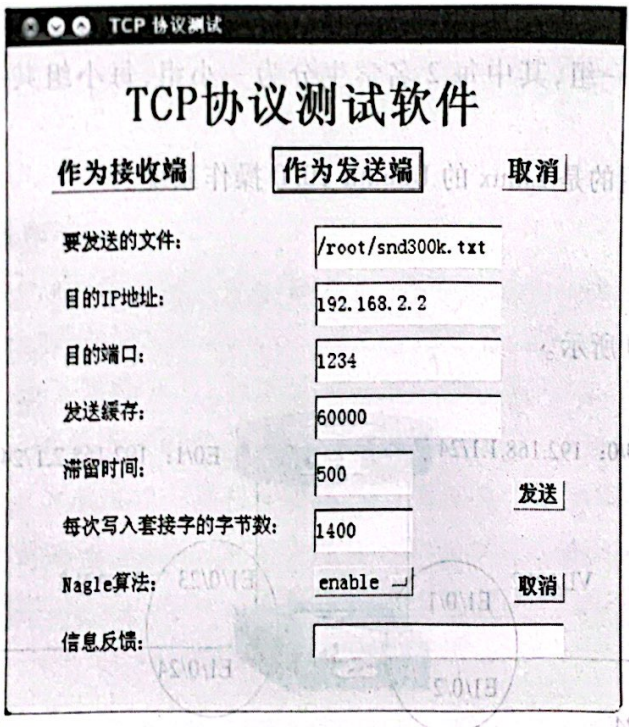



图 4-11　发送端参数设置

文件传输完成后,将截获的报文命名为"send2-学号"和"receive2-学号",保存到本地磁盘/root/DATA/目录下。

**步骤 5**　在 PC A(PC C)(即发送端)的"终端命令行"中运行脚本来读取"TCPConnection 实时监控模块"已记录的此 TCP 连接期间的相关参数数据:

```
root@qjl-desktop:~#  cd /root/TCPLog
root@qjl-desktop:~/TCPLog #  ./read.sh        //点斜杠表示当前目录
```

CS 扫描全能王

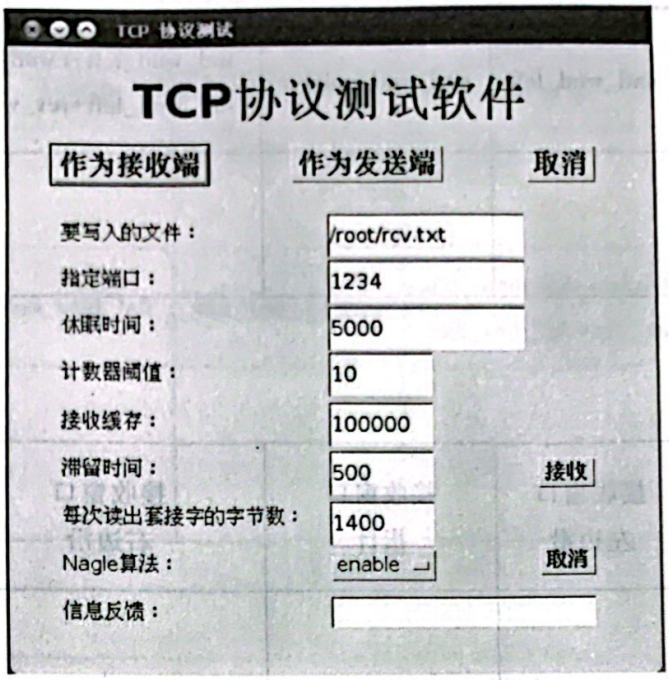



图 4-12 接收端参数设置

此脚本将读取的 TCP 连接相关参数数据保存在本地磁盘/root/TCPLog/目录下的“tcpsndwnddata.txt”和“tcprtodata.txt”文件中，将其复制到/root/DATA/目录下并改名为“send2-学号-tcpsndwnddata.txt”和“send2-学号-tcprtodata.txt”。

**进行分析前请看步骤 7 后面的“注意：*-tcpsndwnddata.txt 和 *-tcprtodata.txt 文件导读”。**

**步骤 6** 分析文件“send2-学号”(或“receive2-学号”)和“send2-学号-tcpsndwnddata.txt”，体会滑动窗口机制。

(1) 分析数据发送部分的前几条报文，描述发送方发送窗口的变化，并解释为什么。

(2) 指出从哪个序号的报文能够明显看出接收端开始休眠，并解释理由。

(3) 分析文件“send2-学号-tcpsndwnddata.txt”，选中 3 次握手连接建立后的前 4 条报文记录(3 条 DATA 报文、一条 ACK 报文，序号为 4、5、6、7)，记下发送方发送窗口的相关值(rcv_wnd，snd_wnd_left，snd_wnd_point，snd_wnd_left+cwnd，snd_wnd_left+rcv_wnd ，(snd_wnd_point- left))。按表 4-4~表 4-6 分析计算接收方(及发送方)的窗口的相关值。

(4) 根据文件“send2-学号-tcpsndwnddata.txt”中发送方的发送窗口相关值进行分析，接收方开始休眠后，描述接收窗口的变化，指出窗口收缩、窗口合拢、窗口张开对应的开始报文序号，并记下“send2-学号-tcpsndwnddata.txt”文件中的对应报文的数值记录(pkt_seqno，pkt_type，…，…)。

CS 扫描全能王

表 4-4　5 号报文(sender----data---->receiver)

| | rcv_wnd | snd_wnd_left | snd_wnd_pointer | snd_wnd_left+cwnd 和<br>snd_wnd_left+rcv_wnd | snd_wnd_point− left |
|---|---|---|---|---|---|
| 发送方<br>发出报文 | | | | | |
| 发送窗口<br>右边沿 | | | | | |
| | 通告的<br>接收窗口 | 接收窗口<br>左边沿 | 接收窗口<br>指针 | 接收窗口<br>右边沿 | 在接收缓存中的数据量<br>(即未确认的数据) |
| 接收方收到<br>DATA 前 | | | | | |
| 接收方收到<br>DATA 后 | | | | | |

表 4-5　6 号报文(sender----data---->receiver)

| | rcv_wnd | snd_wnd_left | snd_wnd_pointer | snd_wnd_left+cwnd 和<br>snd_wnd_left+rcv_wnd | snd_wnd_point− left |
|---|---|---|---|---|---|
| 发送方<br>发出报文 | | | | | |
| 发送窗口<br>右边沿 | | | | | |
| | 通告的<br>接收窗口 | 接收窗口<br>左边沿 | 接收窗口<br>指针 | 接收窗口<br>右边沿 | 在接收缓存中的数据量<br>(即未确认的数据) |
| 接收方收到<br>DATA 前 | | | | | |
| 接收方收到<br>DATA 后 | | | | | |

CS 扫描全能王

表 4-6 7 号报文(receiver----ack---->sender)

| | 通告的接收窗口 | 接收窗口左边沿 | 接收窗口指针 | 接收窗口右边沿 | 在接收缓存中的数据量(即未确认的数据) |
|---|---|---|---|---|---|
| 接收方发出 ACK | | | | | |
| | rcv_wnd | snd_wnd_left | snd_wnd_pointer | snd_wnd_left+cwnd 和 snd_wnd_left+rcv_wnd | snd_wnd_point- left |
| 发送方收到 ACK 后 | | | | | |
| 发送窗口右边沿 | | | | | |
| 发送方收到 ACK 前 | | | | | |
| 发送窗口右边沿 | | | | | |

**步骤 7** 分析文件“send2-组座号”(或“receive2-组座号”),体会窗口侦查机制。

写出窗口侦查开始的报文序号、窗口侦查报文数据长度、窗口侦查报文发送的时间规律。

**注意:* - tcpsndwnddata. txt 和 * -tcprtodata. txt 文件导读。**

在 Linux 系统下查看这些数据文件建议使用 gedit,方法:在图形化视图下选中一个数据文件,右击,在弹出的快捷菜单中选择 Open with gedit 命令,并在“编辑”菜单“首选项”子菜单中取消勾选“启用自动换行”复选项。

如图 4-13 所示,“* - tcpsndwnddata. txt”数据文件中保存的是 TCP 发送方一些与窗口相关的数值记录,具体字段含义如下。

tcpsndwnddata.txt

Show TCP sender's some values about sndwnd:

| pkt_seqno | pkt_type | RorS_seqno | snd_ssthresh | snd_cwnd | rcv_wnd | snd_wnd_left | snd_wnd_pointer | snd_wnd_left+cwnd | snd_wnd_left+rcv_wnd | snd_w |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | snd_con_syn | 1 | 2147483647 | 2 | 0 | 193418421 | 193418422 | 193421341 | 193418421 | 1 |
| 2 | rcv_con_syn_ack | 1 | 2147483647 | 2 | 5792 | 193418422 | 193418422 | 193421342 | 193424214 | 0 |
| 3 | snd_con_ack | 2 | 2147483647 | 3 | 5792 | 193418422 | 193418422 | 193422802 | 193424214 | 0 |
| 4 | snd_data | 3 | 2147483647 | 3 | 5792 | 193418422 | 193419822 | 193422802 | 193424214 | 1400 |
| 5 | snd_data | 4 | 2147483647 | 3 | 5792 | 193418422 | 193421270 | 193422802 | 193424214 | 2848 |
| 6 | snd_data | 5 | 2147483647 | 3 | 5792 | 193418422 | 193422718 | 193422802 | 193424214 | 4296 |
| 7 | rcv_ack | 2 | 2147483647 | 4 | 8688 | 193419822 | 193422718 | 193425662 | 193428510 | 2896 |
| 8 | snd_data | 6 | 2147483647 | 4 | 8688 | 193419822 | 193424166 | 193425662 | 193428510 | 4344 |
| 9 | snd_data | 7 | 2147483647 | 4 | 8688 | 193419822 | 193425614 | 193425662 | 193428510 | 5792 |
| 10 | rcv_ack | 3 | 2147483647 | 5 | 11584 | 193421270 | 193425614 | 193428570 | 193432854 | 4344 |
| 11 | snd_data | 8 | 2147483647 | 5 | 11584 | 193421270 | 193427062 | 193428570 | 193432854 | 5792 |

图 4-13 “* -tcpsndwnddata. txt”示例

CS 扫描全能王

pkt_seqno:此 TCP 连接中所有报文的序号。可与 Wireshark 所截报文对应。所以在 Wireshark 软件截取报文时切记最好使用 TCP 报文过滤。

pkt_type:报文的类型,共有 snd_con_syn、rcv_con_syn_ack、snd_con_ack、snd_data、rcv_ack、timeout、tx_full 等几种类型。

RorS_seqno:发送报文序号或接收报文序号(按发送方发送和接收区分)。

snd_ssthresh:发送方的慢启动阈值。

snd_cwnd:发送方的拥塞窗口大小(以 MSS 为单位)。

rcv_wnd:目前接收方通告的窗口大小(以字节为单位)。

snd_wnd_left:发送窗口左边沿,即已发送数据中等待确认的第一个序号。

snd_wnd_pointer:发送窗口指针,即已发送数据的最高序号+1。

snd_wnd_left+rcv_wnd:此值= snd_wnd_left + rcv_wnd,即发送窗口的左边沿加上接收方通告的接收窗口大小(字节)后的值。

snd_wnd_left+cwnd:此值= snd_wnd_left + snd_cwnd×1 460,即发送窗口的左边沿加上发送方的拥塞窗口大小(字节)后的值。

snd_wnd_pointer-left:此值= snd_wnd_pointer - snd_wnd_left,表明发送方已发送但未被确认的数据字节数。

如图 4-14 所示,"*- tcprtodata. txt"数据文件中保存的是 TCP 发送方一些与超时时间相关的数值记录,具体字段含义如下。

tcprtodata.txt

Show TCP sender's some valuces about RTO:

| pkt_seqno | pkt_type | RorS_seqno | srtt | rto |
|---|---|---|---|---|
| 1 | snd_con_syn | 1 | 0 | 3000 |
| 2 | rcv_con_syn_ack | 1 | 24 | 203 |
| 3 | snd_con_ack | 2 | 24 | 203 |
| 4 | snd_data | 3 | 24 | 203 |
| 5 | snd_data | 4 | 24 | 203 |
| 6 | snd_data | 5 | 24 | 203 |
| 7 | rcv_ack | 2 | 22 | 202 |
| 8 | snd_data | 6 | 22 | 202 |
| 9 | snd_data | 7 | 22 | 202 |

图 4-14　*-tcprttdata. txt 示例

pkt_seqno:此 TCP 连接中所有报文的序号。可与 Wireshark 所截报文对应。

pkt_type:报文的类型,共有 snd_con_syn、rcv_con_syn_ack、snd_con_ack、snd_data、rcv_ack、timeout、tx_full 等几种类型。

RorS_seqno:发送报文序号或接收报文序号(按发送方发送和接收区分)。

srtt:平滑的往返时间(Smoothed Round Trip Time),此值等于 8 000 即为 1 s(为避免内核浮点运算,此处存储的实际上是 srtt 值的 8 倍)。即当每次一个正常的 ACK 报文到达时取样此次往返时间再和以前的平均往返时间加权计算出的平均往返时间。

rto:重传超时(Retransmission Timeout),此值等于 1 000 即为 1 s。

RFC 1122 规定了 RTT 和 RTO 的初始值为:RTT=0 s,RTO=3 s。

CS 扫描全能王

### 3.6.2 慢启动、拥塞避免及拥塞处理和超时与重传机制分析

**步骤 1** 在路由器上配置端口转发速率为 10 Mbps，如下：

```
[Router]interface e0/0
[Router-Ethernet0/0]qos lr outbound cir 10000
[Router-Ethernet0/0]interface e0/1
[Router-Ethernet0/1]qos lr outbound cir 10000
```

**步骤 2** 在 PC A(PC C)(即发送端)和 PC B(PC D)(即接收端)重新开始 Wireshark 报文截获。**注意：启用 TCP 报文过滤。**

**步骤 3** 发送一个 6 MB 的文件，TcpTest 程序参数设置如图 4-15 和图 4-16 所示(**发送端要发送的文件处改成/root/snd6m.txt，接收端休眠时间和计数器阈值均改成 0**)。检查参数设置无误后 PC B(PC D)单击“接收”按钮，然后 PC A(PC C)单击“发送”按钮。**在 PC A(PC C)一单击“发送”按钮后立即 shutdown 路由器一个端口**，如下所示(注意，由于文件发送的时间较快，请预先在 minicom 上输入命令 shutdown，一旦 PC A(PC C)单击“发送”按钮，就立即在 minicom 上按 Enter 键。一定要保证 shutdown 一个路由器端口后，程序还在传送数据。如果 shutdown 一个路由器端口前程序已经传输完数据，则重做此步骤，切记。

```
[Router-Ethernet0/0]shutdown
```

等待 10 s 后再将此端口 undo shutdown：

```
[Router-Ethernet0/0]undo shutdown
```

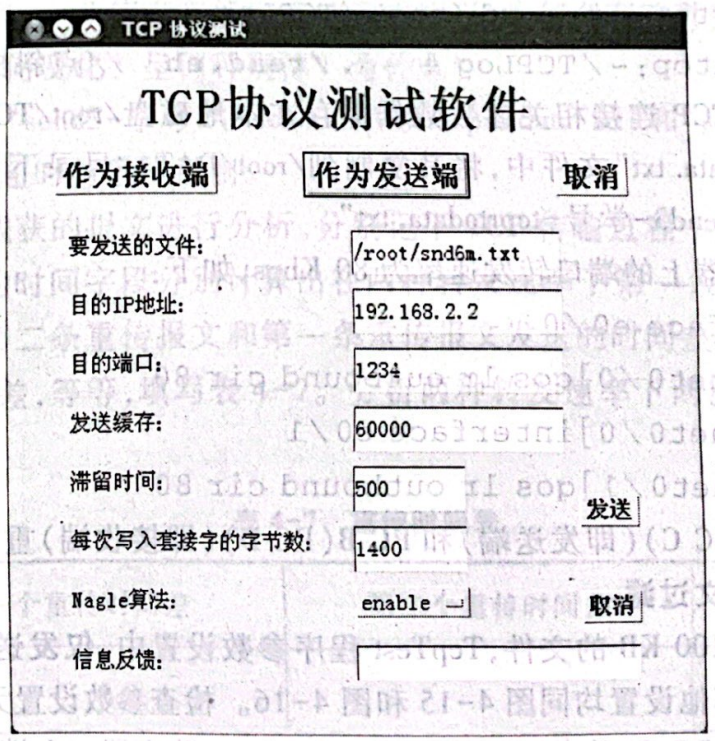



图 4-15 发送端参数设置

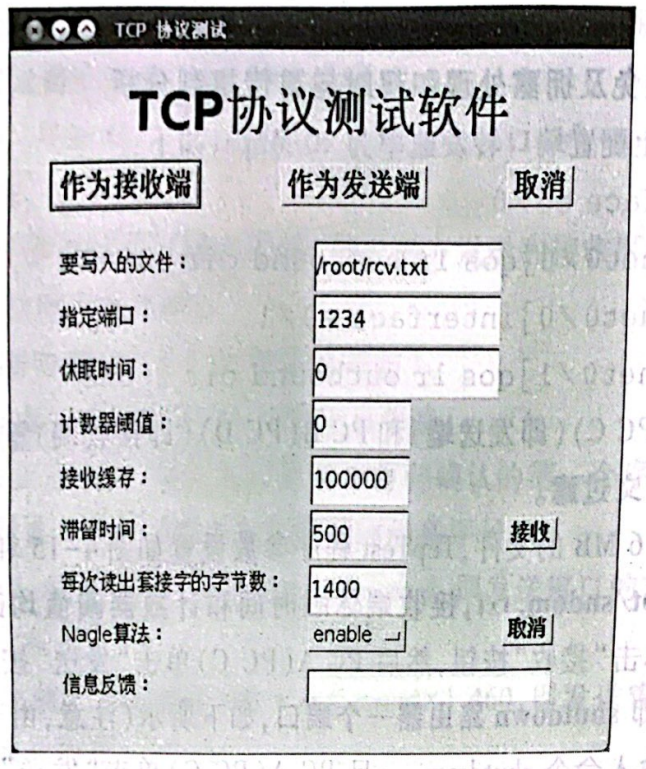



图 4-16　接收端参数设置

文件传输完成后，将截获的报文命名为“send3-学号”和“receive3-学号”，保存到本地磁盘 root/DATA/目录下。

**步骤 4**　在 PC A(PC C)(即发送端)的“终端命令行”中运行脚本来读取“TCPConnection 实时监控模块”已记录的此 TCP 连接期间的相关参数数据：

```
root@qjl-desktop:~#    cd /root /TCPLog
root@qjl-desktop:~/TCPLog #    ./read.sh   //点斜杠表示当前目录
```

此脚本将读取的 TCP 连接相关参数数据保存在本地磁盘/root/TCPLog/目录下的“tcpsndwnddata. txt”和“tcprtodata. txt”文件中，将其复制到/root/DATA/目录下并改名为“send3-学号-tcpsndwnddata. txt”和“send3-学号-tcprtodata. txt”。

**步骤 5**　更改路由器上的端口转发速率为 80 Kbps，如下：

```
[Router]interface e0/0
[Router-Ethernet0/0]qos lr outbound cir 80
[Router-Ethernet0/0]interface e0/1
[Router-Ethernet0/1]qos lr outbound cir 80
```

**步骤 6**　在 PC A(PC C)(即发送端)和 PC B(PC D)(即接收端)重新开始 Wireshark 报文截获。**注意：启用 TCP 报文过滤。**

**步骤 7**　发送一个 100 KB 的文件，TcpTest 程序参数设置中，仅发送端将要发送的文件处改成/root/snd100k. txt，其他设置均同图 4-15 和图 4-16。检查参数设置无误后 PC B(PC D)单击“接收”按钮，然后 PC A(PC C)单击“发送”按钮。shutdown 路由器一个端口(由于此次文件传输较慢，可从容配置命令。但也一定要保证 shutdown 一个路由器端口后，程序还在传送数据，切记)。

CS 扫描全能王

[Router-Ethernet0/0]shutdown

等待40 s后再将此端口undo shutdown：

[Router-Ethernet0/0]undo shutdown

文件传输完成后，将截获的报文命名为“send4-学号”和“receive4-学号”，保存到本地磁盘/root/DATA/目录下。

**步骤8** 在PC A(PC C)(即发送端)的“终端命令行”中运行脚本来读取“TCPConnection实时监控模块”已记录的此TCP连接期间的相关参数数据：

root@qjl-desktop:~# **cd /root/TCPLog**

root@qjl-desktop:~/TCPLog # **./read.sh** //点斜杠表示当前目录

此脚本将读取的TCP连接相关参数数据保存在本地磁盘/root/TCPLog/目录下的“tcpsndwnddata.txt”和“tcprtodata.txt”文件中，将其复制到/root/DATA/目录下并改名为“send4-学号-tcpsndwnddata.txt”和“send4-学号-tcprtodata.txt”。

**步骤9** 分析文件“send3-学号”(或“receive3-学号”)和“send3-学号-tcpsndwnddata.txt”(或“send4-学号”和“send4-学号-tcpsndwnddata.txt”)，体会慢启动、拥塞避免及拥塞处理机制。

(1) 选中第一条发送数据的报文记录，记下其ssthresh和cwnd值是多少，为何为此值？按发送窗口的计算公式计算出当前的发送窗口snd_wnd值，并记下此时的发送窗口左边沿snd_wnd_left值并计算出此时的发送窗口右边沿。

(2) 在随后的发送数据报文中，ssthresh和cwnd值有何变化，呈何种规律？为什么呈此种规律？发送的报文是否可以验证这一规律？

(3) 指出ssthresh和cwnd值有突然变化的报文序号，为何会有这种变化？此后(直至结束)的ssthresh和cwnd值有何变化？呈何种规律？为什么？

**步骤10** 分析文件“send3-学号”、“send3-学号-tcprtodata.txt”和“send4-学号”、“send4-学号-tcprtodata.txt”，体会超时与重传机制。

(1) 对Wireshark截获的报文进行分析，分别记下TCP传输过程中发送第一条重传报文的序号，并请用截获报文的时间字段分别计算出在两种转发速率下第一条重传报文和其对应的原始发送报文的时间差、第二条重传报文和第一条重传报文发送的时间差、第三条重传报文和第二条重传报文发送的时间差，等等，填写表4-7。分析两种转发速率下两次重传的时间差，结合题(2)对此现象做出解释。

表4-7 重传时间差

| | 第一个重传时间差 | 第二个重传时间差 | 第三个重传时间差 |
|---|---|---|---|
| 10 Mbps | | | |
| 80 Kbps | | | |

CS 扫描全能王

(2) 对“send3-学号-tcprtodata. txt”和“send4-学号-tcprtodata. txt”文件进行分析。在正常传输时,两种速率下的 TCP 连接的 RTO 时间分别稳定在多少? 在发生报文超时重传时,RTT 和 RTO 值有何变化? 特别是 RTO 值有何特定变化规律? 为什么? 继续正常报文传输时 RTT 和 RTO 值又有何变化? 为什么?

(3) 课外:查阅相关资料,对 RTT 和 RTO 值的变化做出定量的计算和分析。建议阅读《用 TCP/IP 进行网络互联,第一卷(第四版)》中译本,由美国人 Douglas E. Comer 编著,林瑶等译,电子工业出版社出版。其第 13 章中对 RTT、RTO 等的计算有比较详细的阐述,与 Linux 下对此的实现相同。

### 3.6.3　快重传和快恢复算法分析

**步骤 1**　继续上一小节的实验,配置路由器,取消对接口的速率限制。请写出命令:

**步骤 2**　在 PC B(PC D)(即接收端)的“终端命令行”中装载“TCPConnection 丢包内核模块”:

```
root@qjl-desktop:~#     cd /root /TCPDropPkt
root@qjl-desktop:~/TCPDropPkt #     ./load.sh   //点斜杠表示当前目录
```

**步骤 3**　在 PC A(PC C)(即发送端)和 PC B(PC D)(即接收端)重新开始 Wireshark 报文截获。注意:启用 TCP 报文过滤。

**步骤 4**　发送一个 300 KB 的文件,TcpTest 程序参数设置如图 4-17 和图 4-18 所示(发送方要发送的文件处改成/root/snd300k. txt,接收方休眠时间和计数器阈值均改成 0)。检查参数设置无误后 PC B(PC D)单击“接收”按钮,然后 PC A(PC C)单击“发送”按钮。

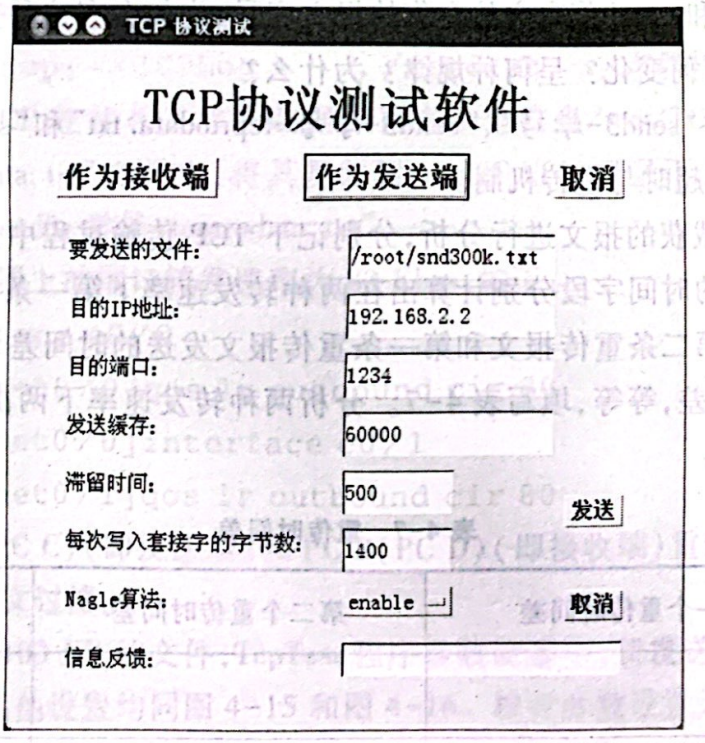



图 4-17　发送端参数设置

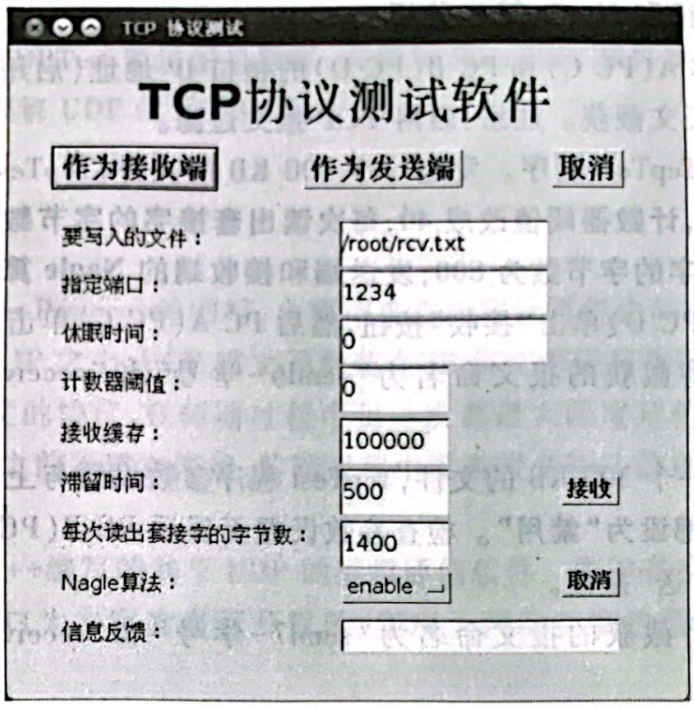



图 4-18 接收端参数设置

文件传输完成后，将截获的报文命名为“send5-学号”和“receive5-学号”，保存到本地磁盘/root/DATA/目录下。

**步骤 5** 在 PC A(PC C)(即发送端)的“终端命令行”中运行脚本来读取“TCPConnection 实时监控模块”已记录的此 TCP 连接期间的相关参数数据：

```
root@qjl-desktop:~#     cd /root /TCPLog
root@qjl-desktop:~/TCPLog #     ./read.sh   //点斜杠表示当前目录
```

此脚本将读取的 TCP 连接相关参数数据保存在本地磁盘/root/TCPLog/目录下的“tcpsndwnddata. txt”和“tcprtodata. txt”文件中，将其复制到/root/DATA/目录下并改名为“send5-学号-tcpsndwnddata. txt”和“send5-学号-tcprtodata. txt”。

**步骤 6** 分析文件“send5-学号”、“receive5-学号”和“send5-学号-tcpsndwnddata. txt”、“send5-学号-tcprtodata. txt”，体会快重传和快恢复算法的作用。

(1) 请写出前 3 个重复 ACK 的报文序号。在第三个重复 ACK 报文到达后，发送报文发生什么变化？为什么？

(2) 在前 3 个重复的 ACK 报文到达期间，ssthresh 和 cwnd 有何变化？RTT 和 RTO 有何变化？为什么？

(3) 在第三个重复 ACK 报文到达后是否还有重复 ACK 报文到达？随之(直至传输结束 ssthresh 和cwnd 有何变化？呈何种规律？为什么有此规律？

CS 扫描全能王

### 3.6.4　糊涂窗口综合征和 Nagle 算法分析

**步骤 1**　配置好 PC A(PC C)和 PC B(PC D)的接口 IP 地址(启用相应的网络连接),并分别启动 Wireshark 进行报文截获。**注意:启用 TCP 报文过滤。**

**步骤 2**　分别启动 TcpTest 程序。发送一个 300 KB 的文件,TcpTest 程序参数设置如下:**接收端休眠时间 5 000 ms,计数器阈值改成 40,每次读出套接字的字节数为 500;发送端发送缓存为 2 000,每次写入套接字的字节数为 800;发送端和接收端的 Nagle 算法都设为"启用"。**检查参数设置无误后 PC B(PC D)单击"接收"按钮,然后 PC A(PC C)单击"发送"按钮。

文件传输完成后,将截获的报文命名为"send6-学号"和"receive6-学号",保存在本地磁盘。

**步骤 3**　重新发送一个 300 KB 的文件,TcpTest 程序参数设置**与上面实验不同的是:发送端和接收端的 Nagle 算法都设为"禁用"。**检查参数设置无误后 PC B(PC D)单击"接收"按钮,然后 PC A(PC C)单击"发送"按钮。

文件传输完成后,将截获的报文命名为"send7-学号"和"receive7-学号",保存在本地磁盘。

**步骤 4**　分析文件"send6-学号"、"receive6-学号"和"send7-学号"、"receive7-学号",体会糊涂窗口综合征和 Nagle 算法的作用。

(1) 分别分析两次 TCP 传输过程中数据发送部分的前面几条报文,结合 TcpTest 程序参数设置,分析其数据长度的变化,并解释为什么。

(2) 在两次 TCP 传输过程中,窗口通告为 0 后,窗口张开时通告的第一个窗口大小分别是多少? 为什么?

(3) 通过这两次 TCP 传输实验,比较分析 Nagle 算法的作用。

最后,将所有实验数据上传 FTP 服务器相应目录。

### 3.7　实验总结

通过实验,初步验证了 TCP 中关于流量控制和拥塞控制的主要算法,体会了滑动窗口机制、糊涂窗口综合征的避免、慢启动、拥塞避免、重传机制、快重传和快恢复等算法的作用,以及这些算法如何有机地结合起来实现高效的面向连接的可靠传输服务。

## 4　UDP 分析

### 4.1　实验目的

(1) 分析 UDP 报文格式。

(2) 了解和理解 UDP 的运行机理及其作用。



UDP 分析

CS 扫描全能王

## 4.2 实验内容

使用模拟通信程序 UDPTest 发送消息数据，并使用 Wireshark 软件截获报文，分析 UDP 的报文格式。并进而了解和理解 UDP 的运行机理。

## 4.3 实验原理

1. UDP 简介

UDP 是 User Datagram Protocol 的缩写，主要用来支持在计算机之间传输数据的网络应用，与 TCP 一样，UDP 也是位于 IP 之上，UDP 报文是封装在 IP 报文中进行传输的。

UDP 是基于数据报文的协议，在传输过程中每一次都最大限度地传输数据。同时，UDP 又是不可靠的，在传输数据之前不建立连接，传输过程中没有报文确认信息。

2. UDPTest 软件介绍

UDPTest 程序是用 C++编写的基于 UDP 的模拟通信软件。程序是在 Microsoft Visual C++平台上，使用标准 Socket 接口为本实验进行开发的，实现了两端互发消息进行通信的功能。程序界面如图 4-19 所示。

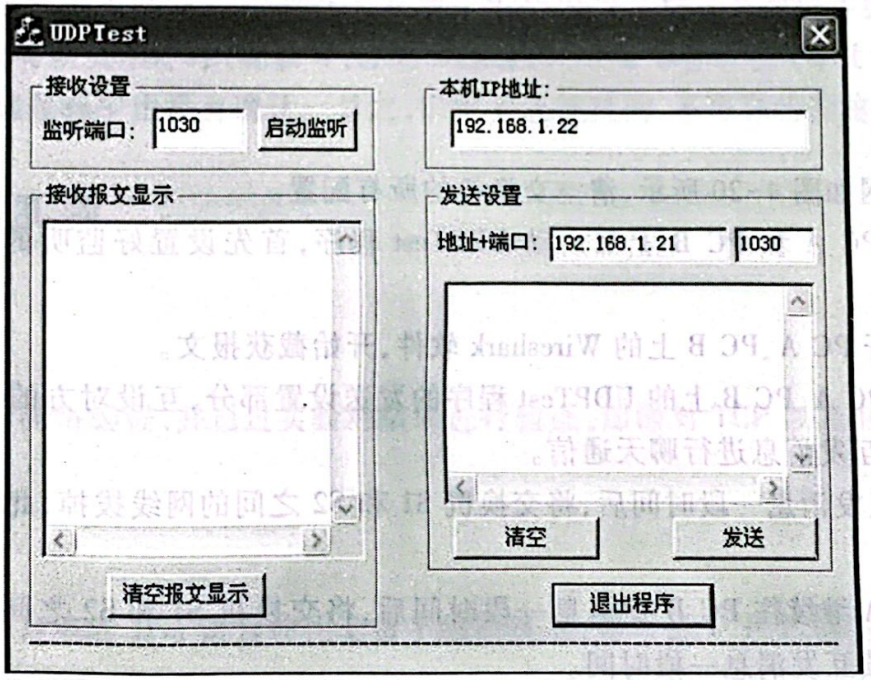



图 4-19 UDPTest 程序界面

在程序启动后，程序会自动显示出本机的 IP 地址。

程序的使用十分简单，只需在启动程序后首先设置好监听的 UDP 端口并启动监听，即可接收任何发往本机被监听 UDP 端口的 UDP 消息。同时还可以设置想要发送消息的目的 IP 地址和目的 UDP 端口，输入消息内容单击“发送”按钮即可发出此消息，当然前提是对方(目的 IP 主机)已用本程序在相应 UDP 端口进行监听。这样即可实现两者之间的消息通信。

CS 扫描全能王

## 4.4 实验环境与分组

(1) 三层交换机 2 台,标准网线 4 条,Console 线 4 条,计算机 4 台。

(2) 每 4 名学生分为一组,共同配合,完成实验。

(3) 本实验 PC 使用 Windows XP 操作系统。

## 4.5 实验组网

实验组网如图 4-20 所示。

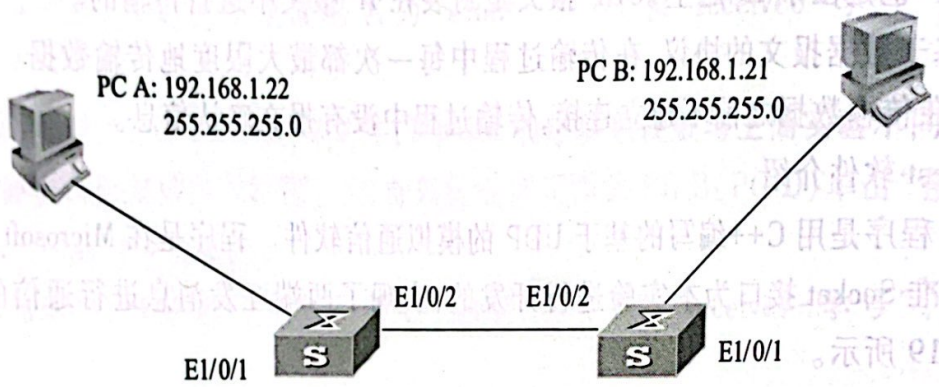



图 4-20 UDP 分析组网图

## 4.6 实验步骤

**步骤 1** 组网如图 4-20 所示,清空交换机的所有配置。

**步骤 2** 在 PC A 和 PC B 上都启动 UDPTest 程序,首先设置好监听的 UDP 端口并启动监听。

**步骤 3** 打开 PC A、PC B 上的 Wireshark 软件,开始截获报文。

**步骤 4** 在 PC A、PC B 上的 UDPTest 程序的发送设置部分,互设对方的 IP 地址和 UDP 监听端口(1030)。互发消息进行聊天通信。

**步骤 5** 在互发消息一段时间后,将交换机 S1 和 S2 之间的网线拔掉,此时 PC A 再继续往 PC B 发消息。

**步骤 6** PC A 继续往 PC B 发消息一段时间后,将交换机 S1 和 S2 之间的网线重新插上。PC A 和 PC B 继续互发消息一段时间。

**步骤 7** 操作 PC A、PC B 上的 Wireshark 软件,分析截获得报文,填写表 4-8。

(1) 将实验截获的报文命名为“UDP-学号”,上传至 FTP 服务器的“网络实验\传输层实验”目录下。

(2) 分析 UDP 报文结构:选中第一个 UDP 报文,将 UDP 协议树中各字段名、字段长度、字段值、字段表达信息填入表 4-8,并绘制 UDP 报文结构,详细绘制 UDP 协议树字段。

(3) UDP 报文结构与 TCP 报文结构有什么区别?

CS 扫描全能王

表 4-8 UDP 协议树中的信息

| 字段名 | 字段长度 | 字段值 | 字段表达信息 |
| --- | --- | --- | --- |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

(4) 在步骤 5 交换机 S1 和 S2 之间的网线拔掉期间,PC A 向 PC B 发送的 UDP 消息,在步骤 6 交换机 S1 和 S2 之间的网线重新插上之后 PC B 是否收到? 请解释为什么会出现这种现象。

(5) 综合分析 TCP 和 UDP 的不同之处。

### 4.7 实验总结

本次实验利用 UDPTest 程序互发 UDP 消息进行通信,利用 Wireshark 截获并分析了 UDP 报文,并设计了一个小实验来验证 UDP 的运行机理。从 UDP 报文格式及验证实验中可以看到,在 UDP 报文的首部没有报文顺序号、确认号,没有 SYN、FIN、ACK 等标志位,所以 UDP 报文的传输没有建立连接,传输过程中也没有确认。总之,UDP 是无连接的、不可靠的传输层协议。

## 5 TCP 编程实验

### 5.1 实验目的

基于 TCP 进行网络编程,并通过实验对结果进行验证,加深对 TCP 以及其他网络基本概念的理解。

### 5.2 实验内容

基于 C/C++语言或者 Java 语言使用套接字编程技术,编写基于 TCP 的程序,来测量网络宽,并通过实验进行验证。

### 5.3 实验原理

从源端发送一个文件到目的端,在发送开始和结束时读取系统时间,得到 TCP 传送文件需要的总时间。详细分析 TCP 传输文件的过程,构造一个算法来计算带宽。为了保证结果靠性可以重复测试多次,取平均值作为最终结果。

具体实现方法请参考有关文献和本实验前面的 TcpTest 程序介绍。

CS 扫描全能王

## 5.4 实验环境与分组

(1) 路由器和交换机各 1 台,标准网线 4 条,Console 线 2 条,计算机 4 台。

(2) 每名同学独立完成。

## 5.5 实验组网

实验组网如图 4-21 所示。

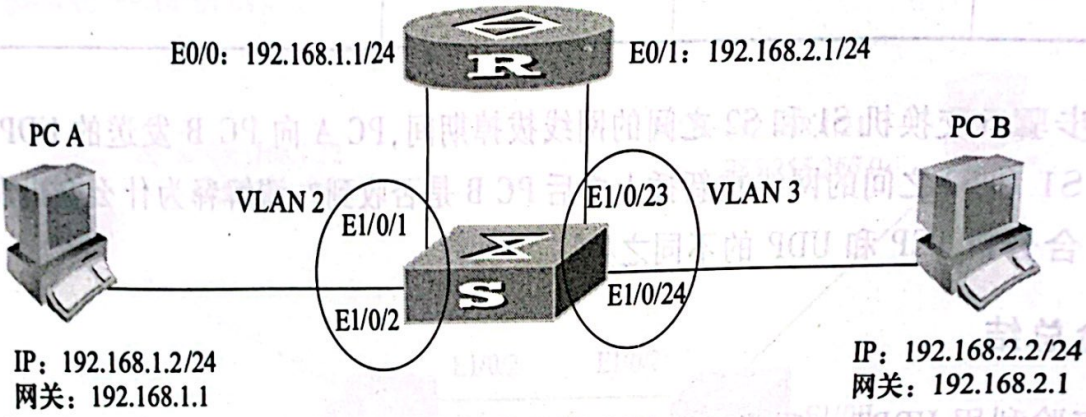



图 4-21 TCP 编程实验组网

## 5.6 实验步骤

**步骤 1** 将编写的程序导入计算机中,编译运行。

**步骤 2** 通过在路由器上进行限速和多台主机同时在网络上传送文件来制造不同的网络环境,观察不同情况下网络带宽的变化,分析测量结果,说明为什么不同情况下测得的带宽会有差异。

**步骤 3** 将编写的程序和对实验结果的分析上传到 FTP 服务器。

## 5.7 实验总结

通过设计和实现具有一定功能的 TCP 应用程序,加深对 TCP 的理解,逐步提高网络编程能力。

# 6 设计型实验

通过以上实验可以知道 UDP 是无连接的、不可靠的传输层协议。那么如何用 UDP 实现可靠的数据传输呢? 请设计基于 C/C++语言或者 Java 语言使用套接字技术,编写基于 UDP 的可靠数据传输程序,并通过实验进行验证。

提示:可参考 TFTP(RFC 1350,TFTP Version 2)的设计和实现。

CS 扫描全能王

## 预习报告

1. 预习 TCP 报文结构，将 TCP 报文首部各字段名、字段长度、字段表达信息，填写表 4-9。字段值请根据 MOOC 网站或 FTP 所提供的报文填写。

表 4-9 TCP 报文结构

| 字段名 | 长度 | 字段值 | | 字段意义 |
|---|---|---|---|---|
| | | 发送报文 | 确认报文 | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

2. 写出 UDP 的报文格式。
3. 简要描述 TCP 连接建立和释放的过程。
4. 简要描述 TCP 的滑动窗口机制。
5. 简要描述慢启动和拥塞避免算法。
6. 简要描述快重传和快恢复算法。
7. 简要描述 TCP 传输中时间延迟的计算，以及超时重传机制。
8. 简要描述糊涂窗口综合征产生的原因及其解决方法。

CS 扫描全能王